تطبيق دوائي — كود Flutter

هذا الملف يحتوي على الكود الذي أرسلته كما هو، موزعًا على صفحات أفقية لتسهيل القراءة.

تنبيه: الاتصال التلقائي قد لا يعمل بمجرد وضع الكود في المشروع؛ يجب إعداد أذونات Android ومراجعة توافق الحزم واختبار السلوك بأمان. لا تختبريه بالاتصال برقم الطوارئ الحقيقي.

```
import 'package:android_alarm_manager_plus/android_alarm_manager_plus.dart';
import 'package:flutter/material.dart';
import 'package:flutter_local_notifications/flutter_local_notifications.dart';
import 'package:flutter_phone_direct_caller/flutter_phone_direct_caller.dart';
import 'package:permission_handler/permission_handler.dart';
import 'package:shared_preferences/shared_preferences.dart';

const green = Color(0xFF2E7D32);
const grayBg = Color(0xFFF1F3F2);
const grayText = Color(0xFF616161);
const escalateAfter = Duration(minutes: 10); // ■■■■■ ■■■■■ ■■■■ ■■■■■■■■

final _np = FlutterLocalNotificationsPlugin();

String _today() => DateTime.now().toIso8601String().substring(0, 10);

Future<void> initNotifs() async {
  await _np.initialize(
    const InitializationSettings(
        android: AndroidInitializationSettings('@mipmap/ic_launcher')),
    onDidReceiveNotificationResponse: _onResp,
    onDidReceiveBackgroundNotificationResponse: _onRespBg,
  );
}

@pragma('vm:entry-point')
void _onRespBg(NotificationResponse r) => _onResp(r);

Future<void> _onResp(NotificationResponse r) async {
  if (r.actionId == 'done' && r.payload != null) {
    final p = await SharedPreferences.getInstance();
    await p.setString('done_${r.payload}', _today());
  }
}

// ■■■■■ ■■■■■■■■ ■■■■ ■■■■■■■
@pragma('vm:entry-point')
Future<void> remind(int alarmId) async {
  WidgetsFlutterBinding.ensureInitialized();
  final medId = alarmId ~/ 2;
  final p = await SharedPreferences.getInstance();
  await p.reload();
  final med = (p.getStringList('meds') ?? [])
      .map((e) => e.split('|'))
      .where((m) => m[0] == '$medId')
      .toList();
  if (med.isEmpty) return;
  await initNotifs();
  await _np.show(
    medId,
    '■■■■ ■■■■■■■ ■',
    med.first[1],
    const NotificationDetails(
      android: AndroidNotificationDetails(
        'meds',
        '■■■■■■ ■■■■■■■',
        importance: Importance.max,
        priority: Priority.high,
        fullScreenIntent: true,
```

```
        actions: [AndroidNotificationAction('done', '■■ ■■■ ■■■■■■')],
      ),
    ),
    payload: '$medId',
  );
}

// ■■■ ■■ ■■ ■■■■■■■■ ■■■ ■■■■■■ ■■■■ ■■■■■■■■
@pragma('vm:entry-point')
```

```
Future<void> escalate(int alarmId) async {
  WidgetsFlutterBinding.ensureInitialized();
  final medId = alarmId ~/ 2;
  final p = await SharedPreferences.getInstance();
  await p.reload();
  if (p.getString('done_$medId') == _today()) return;
  final log = p.getStringList('log_${_today()}') ?? [];
  log.add('■■ ■■■ ■■■■ ■■■ ■■■■■ ■■■■■■ ■■■ $medId - ■■ ■■■■■■■ ■■■■■■■■');
  await p.setStringList('log_${_today()}', log);
  final n = p.getString('emergency') ?? '';
  if (n.isNotEmpty) await FlutterPhoneDirectCaller.callNumber(n);
}

Future<void> scheduleMed(int id, TimeOfDay t) async {
  final now = DateTime.now();
  var at = DateTime(now.year, now.month, now.day, t.hour, t.minute);
  if (at.isBefore(now)) at = at.add(const Duration(days: 1));
  await AndroidAlarmManager.periodic(const Duration(days: 1), id * 2, remind,
      startAt: at, exact: true, wakeup: true, rescheduleOnReboot: true);
  await AndroidAlarmManager.periodic(
      const Duration(days: 1), id * 2 + 1, escalate,
      startAt: at.add(escalateAfter),
      exact: true,
      wakeup: true,
      rescheduleOnReboot: true);
}

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await AndroidAlarmManager.initialize();
  await initNotifs();
  await [Permission.notification, Permission.phone, Permission.scheduleExactAlarm]
      .request();
  final p = await SharedPreferences.getInstance();
  runApp(DawaeiApp(loggedIn: p.getBool('loggedIn') ?? false));
}

class DawaeiApp extends StatelessWidget {
  final bool loggedIn;
  const DawaeiApp({super.key, required this.loggedIn});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: '■■■■■■',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: green),
        scaffoldBackgroundColor: grayBg,
        useMaterial3: true,
        inputDecorationTheme: const InputDecorationTheme(
            filled: true, fillColor: Colors.white, border: OutlineInputBorder()),
      ),
      builder: (c, child) =>
          Directionality(textDirection: TextDirection.rtl, child: child!),
      home: loggedIn ? const SettingsScreen() : const WelcomeScreen(),
    );
  }
}

// ---------- ■■■■■■■ 1: ■■■■■■■■ + ■■■■■■ ■■■■■■■ ----------
class WelcomeScreen extends StatefulWidget {
  const WelcomeScreen({super.key});
  @override
  State<WelcomeScreen> createState() => _WelcomeScreenState();
}

class _WelcomeScreenState extends State<WelcomeScreen> {
```

```
  final email = TextEditingController();
  final pass = TextEditingController();

  Future<void> login() async {
    if (!email.text.contains('@') || pass.text.length < 6) {
      ScaffoldMessenger.of(context).showSnackBar(const SnackBar(
          content: Text('[illegible] [illegible] [illegible] [illegible] [illegible] 6 [illegible] [illegible] [illegible]')));
      return;
    }
    final p = await SharedPreferences.getInstance();
    await p.setBool('loggedIn', true);
    await p.setString('account', email.text.trim());
    if (!mounted) return;
    Navigator.pushReplacement(
        context, MaterialPageRoute(builder: (_) => const SettingsScreen()));
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: SafeArea(
        child: ListView(padding: const EdgeInsets.all(24), children: [
          const SizedBox(height: 24),
          const Icon(Icons.medication_rounded, size: 90, color: green),
          const Text('[illegible]',
              textAlign: TextAlign.center,
              style: TextStyle(
                  fontSize: 36, fontWeight: FontWeight.bold, color: green)),
          const SizedBox(height: 12),
          const Text(
            '[illegible] [illegible] [illegible] [illegible] [
```

```
  @override
  void initState() {
    super.initState();
    SharedPreferences.getInstance().then((p) {
      emergency.text = p.getString('emergency') ?? '';
      beneficiary.text = p.getString('beneficiary') ?? '';
      reportEmail.text = p.getString('reportEmail') ?? '';
    });
  }

  Future<void> save() async {
    final p = await SharedPreferences.getInstance();
    await p.setString('emergency', emergency.text.trim());
    await p.setString('beneficiary', beneficiary.text.trim());
    await p.setString('reportEmail', reportEmail.text.trim());
    if (!mounted) return;
    Navigator.push(
        context, MaterialPageRoute(builder: (_) => const MedsScreen()));
  }

  Widget field(TextEditingController c, String label, IconData i,
          TextInputType t) =>
      Padding(
        padding: const EdgeInsets.only(bottom: 14),
        child: TextField(
            controller: c,
            keyboardType: t,
            decoration: InputDecoration(labelText: label, prefixIcon: Icon(i))),
      );

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
          title: const Text('[redacted]'),
          backgroundColor: green,
          foregroundColor: Colors.white),
      body: ListView(padding: const EdgeInsets.all(20), children: [
        field(emergency, '[redacted]', Icons.emergency, TextInputType.phone),
        field(beneficiary, '[redacted]', Icons.person, TextInputType.phone),
        field(reportEmail, '[redacted]', Icons.email,
            TextInputType.emailAddress),
        const SizedBox(height: 10),
        FilledButton(
            style: FilledButton.styleFrom(
                backgroundColor: green, minimumSize: const Size(0, 52)),
            onPressed: save,
            child: const Text('[redacted]', style: TextStyle(fontSize: 18))),
      ]),
    );
  }
}

// ---------- [redacted] 3: [redacted] [redacted] ----------
class MedsScreen extends StatefulWidget {
  const MedsScreen({super.key});
  @override
  State<MedsScreen> createState() => _MedsScreenState();
}

class _MedsScreenState extends State<MedsScreen> {
  List<List<String>> meds = []; // [id, name, HH:mm]

  @override
  void initState() {
    super.initState();
    load();
```

```
}

Future<void> load() async {
  final p = await SharedPreferences.getInstance();
  setState(() => meds =
      (p.getStringList('meds') ?? []).map((e) => e.split('|')).toList());
}

Future<void> saveList() async {
  final p = await SharedPreferences.getInstance();
  await p.setStringList('meds', meds.map((m) => m.join('|')).toList());
}

Future<void> add() async {
  final name = TextEditingController();
  final ok = await showDialog<bool>(
    context: context,
    builder: (c) => AlertDialog(
      title: const Text('[redacted]'),
      content: TextField(
          controller: name,
          decoration: const InputDecoration(labelText: '[redacted]')),
      actions: [
        TextButton(
            onPressed: () => Navigator.pop(c, false),
            child: const Text('[redacted]')),
        FilledButton(
            onPressed: () => Navigator.pop(c, true),
            child: const Text('[redacted]')),
      ],
    ),
  );
  if (ok != true || name.text.trim().isEmpty || !mounted) return;
  final t =
      await showTimePicker(context: context, initialTime: TimeOfDay.now());
  if (t == null) return;
  final id = DateTime.now().millisecondsSinceEpoch ~/ 1000 % 100000;
  meds.add([
    '$id',
    name.text.trim(),
    '${t.hour.toString().padLeft(2, '0')}:${t.minute.toString().padLeft(2, '0')}'
  ]);
  await saveList();
  await scheduleMed(id, t);
  setState(() {});
}

Future<void> remove(int i) async {
  final id = int.parse(meds[i][0]);
  await AndroidAlarmManager.cancel(id * 2);
  await AndroidAlarmManager.cancel(id * 2 + 1);
  meds.removeAt(i);
  await saveList();
  setState(() {});
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
        title: const Text('[redacted]'),
        backgroundColor: green,
        foregroundColor: Colors.white),
    floatingActionButton: FloatingActionButton.extended(
        backgroundColor: green,
        foregroundColor: Colors.white,
        onPressed: add,
        icon: const Icon(Icons.add),
```

```
        label: const Text('■■■■■■ ■■■■■')),
      body: meds.isEmpty
          ? const Center(
              child: Text('■■ ■■■■■ ■■ ■■■■■ ■■■■',
                  style: TextStyle(color: grayText, fontSize: 18)))
          : ListView.builder(
              padding: const EdgeInsets.all(16),
              itemCount: meds.length,
              itemBuilder: (c, i) => Card(
                child: ListTile(
                  leading: const Icon(Icons.medication, color: green, size: 34),
                  title: Text(meds[i][1],
                      style: const TextStyle(
                          fontSize: 20, fontWeight: FontWeight.w600)),
                  subtitle: Text('■■ ■■■■ ■■■■■■■ ${meds[i][2]}'),
                  trailing: IconButton(
                      icon: const Icon(Icons.delete_outline, color: grayText),
                      onPressed: () => remove(i)),
                ),
              ),
            ),
    );
  }
}
```